SONARQUBE

SAST

Static Application Security Testing

UJIAN TENGAH SEMESTER · DEVSECOPS

Audit Keamanan STT-NF Blog Platform

PHP / LARAVEL

SONARQUBE COMMUNITY BUILD 26.9

KELOMPOK 6

Firmansyah Dzakwan Arifien · 0110223282
Azzam Fikri · 0110225110
Dhiya Nailah Rizqya · 0110225095
Ghifana Adzkia Ahmad · 0110225044
Rizky Putri · 0110225064
Muhammad Fathir Lubis · 0110225005
Muhammad Bintang Siregar · 0110225058

Teknik Informatika · STT Terpadu Nurul Fikri · Dosen: April Rustianto, S.Komp., M.T., CCIE

PENDAHULUAN

SAST dan SonarQube

Apa itu SAST?

Pengujian keamanan dengan menganalisis source code tanpa menjalankan aplikasi (white-box). Kode di-parse lalu dicocokkan dengan rule pola kode tidak aman. Cepat, bisa diotomasi di CI/CD, dan menemukan masalah sejak tahap development (shift left).

Apa itu SonarQube?

Platform inspeksi kualitas dan keamanan kode. SonarScanner menganalisis kode lalu mengirim hasil ke server. Issue dikelompokkan per software quality (Security, Reliability, Maintainability) dengan severity Blocker sampai Info, dipetakan ke OWASP dan CWE, dan dinilai oleh Quality Gate.

STUDI KASUS

STT-NF Blog Platform (kelompok-A-blog-platform): aplikasi blog Laravel dengan 4 controller, 2 model, middleware CSRF, dan konfigurasi database. 140 baris kode PHP.

METODOLOGI

Alur Audit: Scan, Analisis, Perbaiki, Re-scan

01

Scan awal

Source asli di-scan ke project uts-kelompok6-blog-sebelum

02

Analisis

Setiap issue: severity, file & baris, OWASP, dampak + review manual

03

Perbaikan

Kode diperbaiki, tiap perbaikan jadi commit git terpisah

04

Re-scan

Kode hasil perbaikan di-scan ke project uts-kelompok6-blog-sesudah

SERVER

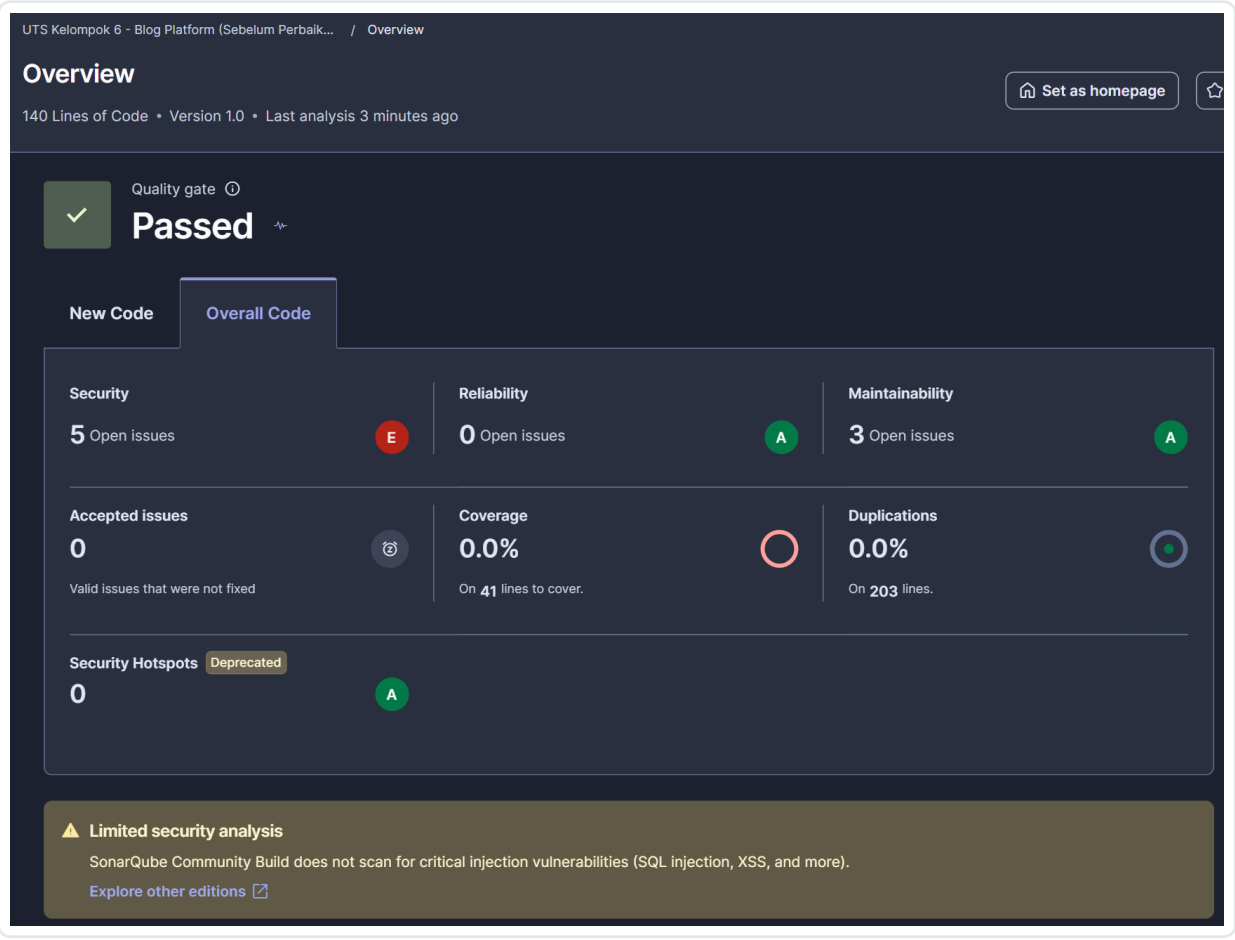
sonarqube.cyphera.my.id · Community Build v26.9.0

SCANNER

SonarScanner CLI 7.2.0 · Profile & Gate "Sonar way"

```
sonar-scanner \
  -Dsonar.host.url=https://sonarqube.cyphera.my.id \
  -Dsonar.token=<ANALYSIS_TOKEN> \
  -Dsonar.projectKey=uts-kelompok6-blog-sebelum \
  -Dsonar.sources=app,routes,config
```

Dashboard Sebelum Perbaikan



Dashboard project uts-kelompok6-blog-sebelum

8 TOTAL ISSUE

5 SECURITY

E SECURITY RATING

LIMITED SECURITY ANALYSIS

Community Build tidak menjalankan taint analysis, jadi SQL Injection dan sejenisnya tidak dilaporkan.

RINGKASAN TEMUAN

Jumlah per Kategori per Severity

Kategori	Blocker	High	Medium	Low	Info	Total
Security	1	3	0	1	0	**5**
Reliability	0	0	0	0	0	**0**
Maintainability	0	0	2	0	1	**3**
Total	**1**	**3**	**2**	**1**	**1**	**8**

Klasifikasi lama: 4 Vulnerability (1 Blocker, 3 Critical) · 4 Code Smell · 0 Bug · 0 Security Hotspot

4 TEMUAN SECURITY UTAMA

BLOCKER

Google API key hardcoded

`AdminController.php : 9`

HIGH

Weak hash MD5 (2x)

`AuthController.php : 12, 25`

HIGH

CSRF dinonaktifkan

`VerifyCsrfToken.php : 12`

LOW

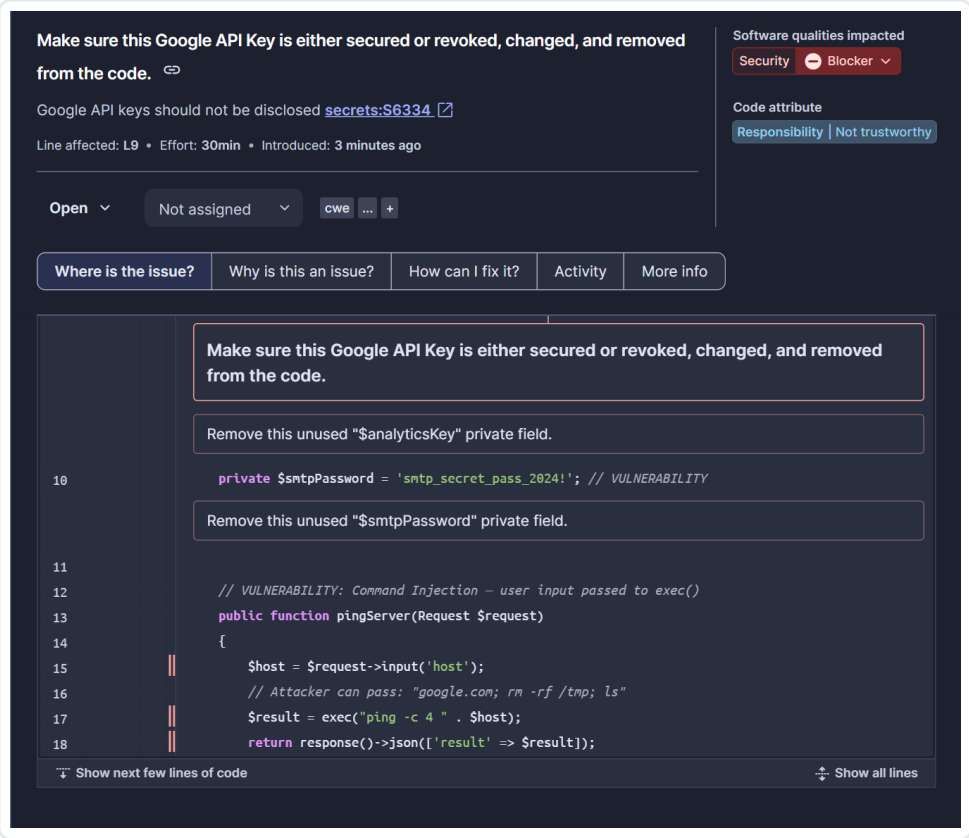
Hardcoded IP address

`config/database.php : 15`

Hardcoded Google API Key & Password SMTP

BLOCKER

AdminController.php baris 9-10 · OWASP A07:2021 · CWE-798




```
// Hardcoded API key for analytics
private $analyticsKey = 'AIzaSyD-9tSr****';
private $smtpPassword = 'smtp_secret_pass_2024!';
```

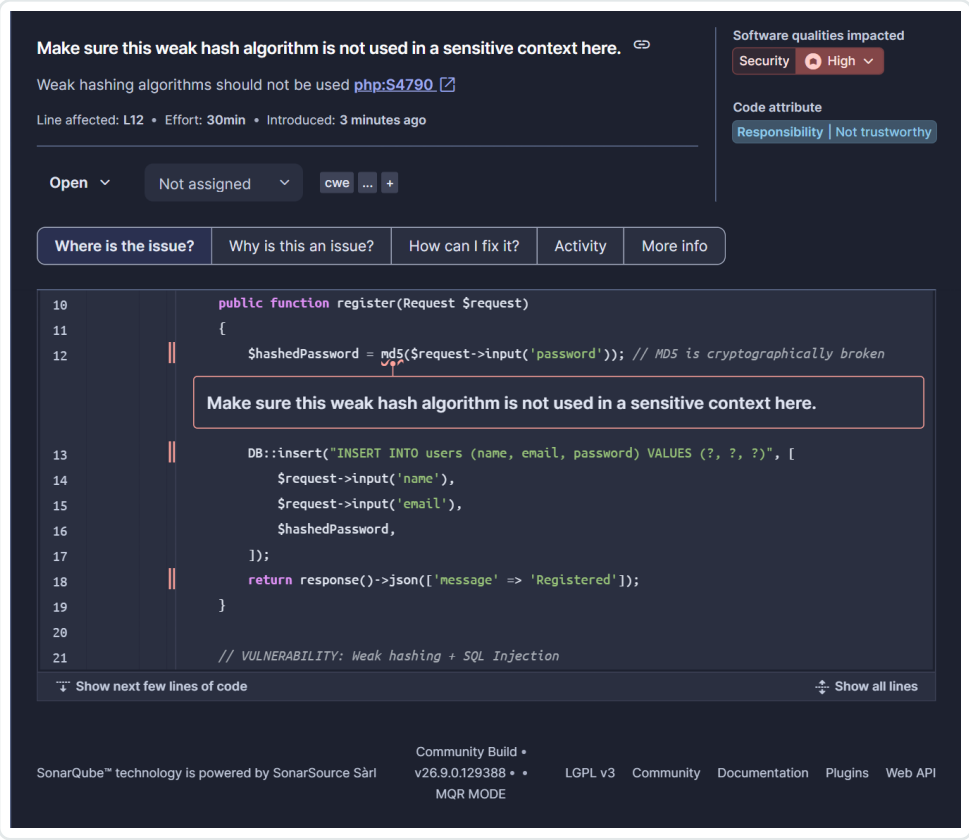
MENGAPA BERBAHAYA

- Siapa pun yang punya akses repositori (termasuk riwayat git) mendapatkan secret.
- API key bisa disalahgunakan atas nama perusahaan; SMTP untuk phishing.
- Secret yang sudah ter-commit harus dirotasi.

Password Di-hash dengan MD5

HIGH

AuthController.php baris 12 & 25 · OWASP A02:2021 · CWE-1240




```php
$hashedPassword = md5($request->input('password'));
...
$user = DB::select("SELECT * FROM users
  WHERE email='$email' AND password='$password'");
```

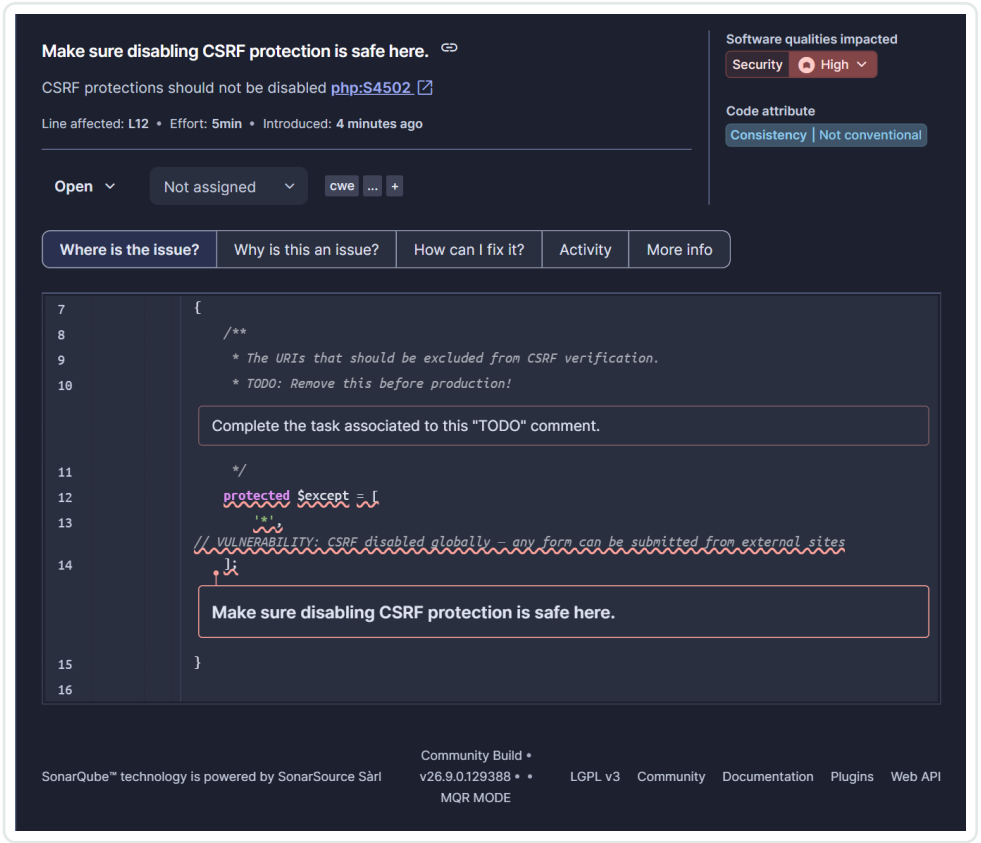
MENGAPA BERBAHAYA

- MD5 cepat dan tanpa salt: hash bocor mudah dipecahkan dengan GPU / rainbow table.
- Password sama menghasilkan hash sama.
- Query login juga rentan SQL Injection (tidak dilaporkan SonarQube).

Proteksi CSRF Dimatikan untuk Semua Route

HIGH

VerifyCsrfToken.php baris 12 · OWASP A01:2021 · CWE-352




```
 * TODO: Remove this before production!
protected $except = [
    '*',
];
```

MENGAPA BERBAHAYA

- Token CSRF tidak pernah diperiksa di route mana pun.
- Halaman jahat bisa mengirim form atas nama korban yang sedang login (hapus post, aksi admin).
- Konfigurasi "sementara" yang terbawa ke production.

ANALISIS 4 · PHP:S1313 + REVIEW MANUAL

Yang Tidak Terlihat oleh SonarQube

LOW config/database.php:15, IP 10.0.0.5 hardcoded. Password DB di file yang sama tidak dilaporkan.

TEMUAN REVIEW MANUAL (TIDAK TERDETEKSI COMMUNITY BUILD)

SQL Injection	ArticleController 14, 24 · AuthController 26	OWASP A03
OS Command Injection	AdminController 17, 27 (exec, shell_exec)	OWASP A03
Path Traversal	FileController 13, 21 (../../.env)	OWASP A01
IDOR	ArticleController 33 (hapus post orang lain)	OWASP A01
Mass Assignment	Post::create($request->all()), $guarded = []	OWASP A08

Pelajaran: 4 vulnerability di dashboard bukan gambaran risiko sebenarnya. SAST tetap perlu review manual / taint analysis / DAST.

REMEDIASI

Sebelum vs Sesudah Perbaikan

Secret ke environment

```
- private $analyticsKey = 'AIza...';
- private $smtpPassword = 'smtp_...';
+ 'key' => env('ANALYTICS_API_KEY'),
```

MD5 ke bcrypt + binding

```
- md5($request->input('password'))
+ Hash::make($validated['password'])
+ DB::selectOne("... WHERE email = ?", [...])
+ Hash::check($pass, $user->password)
```

CSRF aktif kembali

```
- protected $except = ['*'];
+ protected $except = [];
```

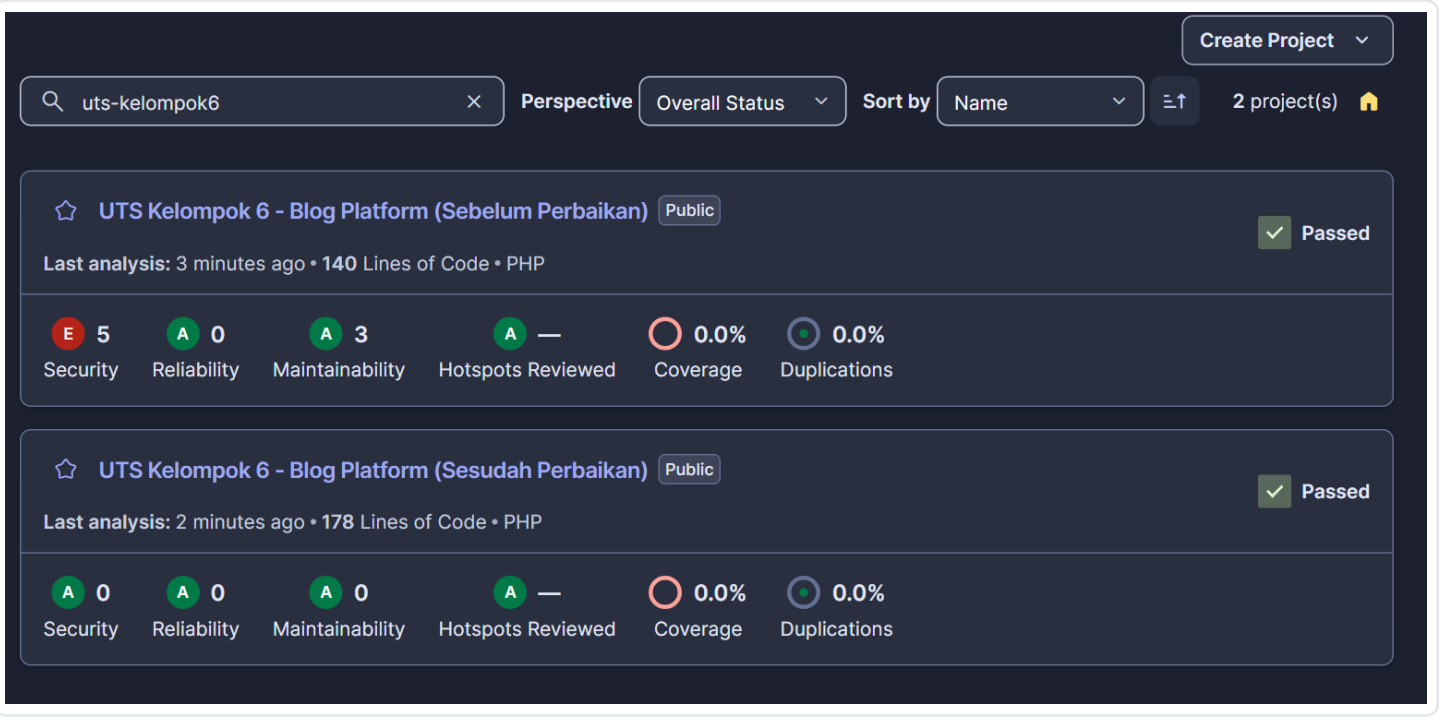
Konfigurasi DB dari .env

```
- 'password' => 'Admin@1234!',
- 'host' => '10.0.0.5',
+ 'password' => env('DB_PASSWORD'),
+ 'host' => env('DB_BACKUP_HOST'),
```

Tambahan dari review manual: parameter binding, whitelist kolom sort, Symfony Process tanpa shell, realpath() untuk file, cek pemilik post, $fillable.

VERIFIKASI RE-SCAN

Dua Project Terpisah: Sebelum vs Sesudah



Project sebelum (Security E, 5 issue) dan sesudah perbaikan (Security A, 0 issue)

Metrik	Sebelum	Sesudah
Security issues	5	0
Security Rating	E	A
Vulnerability	4	0
Maintainability	3	0
Total issue	8	0

HASIL

Semua 8 issue SonarQube teratasi. Kedua project tetap tersimpan sebagai bukti, tidak saling menimpa.

KESIMPULAN

Pelajaran & Rekomendasi

Pelajaran

- Dashboard bersih belum tentu aman: Quality Gate sudah Passed bahkan sebelum perbaikan.
- Community Build tanpa taint analysis: celah injeksi paling kritis justru tidak terlihat.
- Secret yang pernah di-commit harus dianggap bocor dan dirotasi.
- Hampir semua celah muncul karena fitur keamanan Laravel dimatikan atau dilewati.

Rekomendasi

- SonarQube di CI/CD (GitHub Actions) + Quality Gate sebagai syarat merge.
- Lengkapi dengan taint analysis / Semgrep, DAST (OWASP ZAP), dan composer audit.
- Secret di .env / secret manager + gitleaks pre-commit + push protection.
- Standar coding aman: Eloquent/binding, validate(), $fillable, Policy, tanpa exec.
- Tambah unit test agar syarat coverage Quality Gate terpenuhi.

PENGECEKAN OTOMATIS

SonarQube di GitHub Actions

1 Push / Pull Request
Ada perubahan source aplikasi, atau Run manual

2 Checkout
actions/checkout@v4 dengan riwayat git penuh

3 SonarQube Scan
sonarqube-scan-action@v8.3.0, token dari GitHub Secrets

4 Quality Gate
Gagal = job merah, kode tidak aman tertahan sebelum merge

Potongan workflow · sast-sonarqube.yml

```
on: push · pull_request · workflow_dispatch
paths: UTS_Kelompok-6_SAST/kelompok-A-blog-platform/**
uses: SonarSource/sonarqube-scan-action@v8.3.0
SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
-Dsonar.projectKey=uts-kelompok6-blog-ci
-Dsonar.qualitygate.wait=true
```

Log run pertama · GitHub Actions

```
INFO  Project key: uts-kelompok6-blog-ci
INFO  10 files indexed
INFO  Check Quality Gate status
INFO  QUALITY GATE STATUS: PASSED
INFO  EXECUTION SUCCESS
# Checkout ✓  SonarQube Scan ✓  Complete job ✓
```

Hasil CI masuk ke project SonarQube terpisah **uts-kelompok6-blog-ci**, jadi project sebelum/sesudah tetap utuh sebagai bukti.

Terima Kasih

Sesi tanya jawab · Kelompok 6

github.com/FirmansyahDzakwanArifien/DevSecOPS · branch uts/sast-sonarqube-ci